JavaScript Quirks: Why These “Weird” Results Happen

Introduction

JavaScript is a practical language designed for the web. A lot of its “quirks” come from two big goals: backward compatibility (old code must keep working) and convenience (values can be coerced when you use operators like `+`, `==`, and unary `+`).

> **Instructor note:** Most confusing behavior is not random. It follows rules like *type conversion* (ToNumber / ToString / ToPrimitive) and the specific algorithm for *loose equality* (`==`).

Key Concepts (The “Mental Model”)

- **JS has one Number type** (IEEE-754 double precision floating point).
- **Operators trigger coercion**: `+` can mean addition or string concatenation.
- `==` **does conversions**; `===` **does not** (it compares types too).
- **Objects and arrays become primitives** via `valueOf` / `toString` (ToPrimitive).

---

1) `typeof NaN` is `"number"`

`NaN` means “Not-a-Number”, but it is still a value of the *Number* type. Think of it as “an invalid numeric result” (for example, `0 / 0`).

```
typeof NaN
Number.isNaN(NaN)
NaN === NaN
```

> **Output:**

```
"number"
true
false
```

Best practice: use `Number.isNaN(x)` (not `x === NaN`).

---

2) Big integers get rounded: `9999999999999999` becomes `10000000000000000`

JavaScript numbers have limited integer precision. Beyond `Number.MAX_SAFE_INTEGER` (= `9007199254740991`), some integers cannot be represented exactly.

```
Number.MAX_SAFE_INTEGER
9999999999999999
9999999999999999 === 10000000000000000
```

Output:

```
9007199254740991
10000000000000000
true
```

Best practice: if you need exact large integers, use `BigInt`.

```
9999999999999999n
9999999999999999n === 10000000000000000n
```

Output:

```
9999999999999999n
false
```

3) Floating-point math surprises

A) `0.5 + 0.1 === 0.6` is `true`

Some decimals are representable closely enough that comparisons happen to work.

```
0.5 + 0.1
0.5 + 0.1 === 0.6
```

Output:

```
0.6
true
```

B) `0.1 + 0.2 === 0.3` is `false`

`0.1` and `0.2` can't be represented exactly in binary floating point, so the result is a tiny bit off.

```
0.1 + 0.2
0.1 + 0.2 === 0.3
```

Output:

```
0.30000000000000004
false
```

Best practice: compare floats using an epsilon (tolerance) instead of direct equality.

```
function nearlyEqual(a, b, epsilon = 1e-10) {
  return Math.abs(a - b) < epsilon;
}

nearlyEqual(0.1 + 0.2, 0.3)
```

Output:

```
true
```

---

4) `Math.max()` and `Math.min()` with no arguments

These functions are defined to return identity values when called with no parameters:

- `Math.max()` → `-Infinity` (because any real number is bigger than `-Infinity`)
- `Math.min()` → `Infinity` (because any real number is smaller than `Infinity`)

```
Math.max()
Math.min()
```

Output:

```
-Infinity
Infinity
```

> **Tip:** When doing `Math.max(...arr)` on possibly empty arrays, guard it: if the array is empty, handle that case explicitly.

---

5) Arrays/Objects and the `+` operator

`+` is special: if either side becomes a string, JavaScript concatenates strings. Arrays and objects first convert to primitives (often strings).

A) `[] + []` → `""`

```
[] + []
```

Output:

```
""
```

Reason: `[].toString()` is `""`, so it becomes `"" + ""`.

B) `[] + {}` → `"[object Object]"`

```
[] + {}
```

Output:

```
"[object Object]"
```

Reason: `[]` becomes `""`, and `{}.toString()` becomes `"[object Object]"`.

C) `{} + []` → `0` (often)

This one depends on *parsing context*. At the start of a statement, `{}` can be parsed as an empty block, and then unary `+` may apply to `[]` after coercion.

```
{} + []
({} + [])
```

Output:

```
0
"[object Object]"
```

> **Takeaway:** Avoid relying on implicit coercion like this. Be explicit: use `String(x)`, `Number(x)`, or template strings.

---

6) Booleans act like numbers in math

In numeric contexts: `true` becomes `1`, `false` becomes `0`.

```
true + true + true
true + true + true === 3
true - true
```

Output:

```
3
true
0
```

Best practice: if you're using booleans as flags, keep it boolean; don't mix with arithmetic unless you mean to.

---

7) `==` vs `===` (and why `[] == 0` is true)

A) `true == 1` is `true`, but `true === 1` is `false`

```
true == 1
true === 1
```

Output:

```
true
false
```

With `==`, JavaScript converts types to try to compare "equivalent" values. With `===`, types must match.

B) `[] == 0` is `true`

This is the classic coercion chain: `[]` (object) → ToPrimitive → `""` → ToNumber → `0`.

```
[] == 0
[] == ""
Number([])
String([])
```

Output:

```
true
true
0
""
```

> **Rule of thumb:** Prefer `===` and `!==`. Use `==` only when you fully understand the conversion rules.

---

8) The "weird" string from `! + [] + [] + ![]`

This expression is confusing on purpose, but it's a great coercion demonstration.

1. `[]` is truthy → `![]` is `false`
2. `+[]` uses unary plus → converts to number → `0`
3. `!0` is `true`
4. `![]` again gives `false`
5. `true + []` triggers string concatenation after ToPrimitive → `"true"`
6. then `"true" + "" + "false"` → `"truefalse"`

```
!+[]+[]+![]
(!+[]+[]+![]).length
```

Output:

```
"truefalse"
9
```

Takeaway: write readable code; don't be clever with coercion. Debugging time costs more than typing.

---

9) `+` vs `-` with strings

A) `9 + "1"` → `"91"`

If either side is a string (or becomes one), `+` concatenates.

```
9 + "1"
```

Output:

```
"91"
```

B) `91 - "1"` → `90`

`-` is purely numeric, so both sides are converted to numbers.

```
91 - "1"
```

Output:

```
90
```

Best practice: explicitly parse when reading user input: `Number(value)` or `parseInt(value, 10)` when you expect integers.

Quick Reference Table

Expression	Result	Main rule
`typeof NaN`	`"number"`	`NaN` is a Number value
`0.1 + 0.2 === 0.3`	`false`	Floating-point precision
`Math.max()`	`-Infinity`	Identity value for max
`[] + []`	`""`	Arrays stringify to `""`
`true == 1`	`true`	`==` performs coercion
`true === 1`	`false`	`===` compares type + value
`[] == 0`	`true`	Object → primitive → number
`9 + "1"`	`"91"`	`+` concatenates with strings
`91 - "1"`	`90`	`-` forces numeric conversion

Summary

- Most “quirks” are consequences of well-defined coercion rules.
- Prefer `===` / `!==`, and use explicit conversions (`Number()` , `String()`).
- Use epsilon comparisons for decimals; use `BigInt` for large integers.
- Write readable code—clever coercion tricks are fun, but costly in real projects.

Lecture notes: JS Quirks (based on common console examples and coercion rules)